

Alemeno Internship Assignment: Saner

Project summary, workflow, results, and achievements

1. Summary

Saner is an Android React Native marker-scanning app built for the Alemeno internship assignment. It captures a camera snapshot, isolates the marker region, normalizes the output to 300x300, validates the border and anchor structure, and stores accepted markers in the app gallery.

- Built with React Native, Expo, and react-native-vision-camera.
- Targets Android and was tested through a native Android build flow.
- Designed to accept the provided correct markers and reject the provided incorrect markers.
- Uses a structured scanner pipeline instead of matching one specific center illustration.

2. Detailed Explanation

The core challenge was not only finding a square on screen, but finding the correct square reliably when the marker is rotated, slightly off-center, or visually noisy. Earlier heuristic versions were hard to maintain, so the scanner was reorganized into smaller modules and the final logic was centered around three structural cues: the border, the corner anchor, and the normalized marker crop.

- The capture flow is snapshot-based, which keeps the implementation simple and stable on a real device.
- A blob-first crop reduces the amount of irrelevant background sent into the analyzer.
- Ring-based border detection is more rotation-tolerant than checking four rigid strips alone.
- Corner-anchor logic distinguishes valid corner anchors from oversized or misplaced dark blocks.
- The processed output remains fixed at 300x300 to satisfy the assignment requirement exactly.

3. Workflow

- Open the camera preview.
- Align the candidate marker inside the guide frame.
- Tap Capture & Analyze.
- Capture a native snapshot.
- Apply guide crop and dark-blob tightening.
- Normalize the cropped marker to 300x300.
- Run ring-border analysis and corner-anchor analysis.
- If valid, store the processed crop in the accepted gallery.

4. Results

The following reference cases were used as the practical result baseline. The grayscale column is included to show that the detector is primarily structure-driven rather than dependent on the full original color image.

Case	Reference	Grayscale	Expected Result
Correct pig marker			Accepted Borders 4/4, Dark corners 1
Correct dog marker			Accepted Borders 4/4, Dark corners 1
Rotated correct marker			Accepted Accepted despite tilt
Incorrect center-square marker			Rejected No anchor corner found
Incorrect oversized-anchor marker			Rejected Anchor square is oversized

5. Achievements

- Created a working Android marker scanner with native camera capture.
- Built a reusable scanner core split into small, human-readable files.
- Generated exact 300x300 processed outputs.
- Produced a release APK for device installation.
- Documented a visual validation baseline with both original and grayscale reference images.

6. Main Files

- app/camera.tsx: camera UI, capture flow, and gallery.
- app/marker-debug.tsx: reference sample browsing screen.
- src/features/scanner/analyzeCapturedMarker.ts: public scanner API.
- src/features/scanner/markerAnalyzerCore.ts: blob crop, normalized analysis, and scoring.

- `src/features/scanner/markerPixelUtils.ts`: pixel sampling helpers.
- `src/features/scanner/markerPreview.ts`: grayscale preview generation.
- `src/features/scanner/markerAnalysis.types.ts`: shared types and constants.

7. Limitations and Future Improvement

- The scanner is snapshot-based rather than a continuous frame-processor detector.
- The detector is still heuristic and tuned to the provided marker family.
- A full perspective-warp pipeline could improve extreme-angle robustness further.